大家好，我是 Guide！在 AI 应用开发中，用户体验至关重要。当 LLM 需要较长时间生成回答时，传统的"等待-返回"模式会让用户感到焦虑。SSE（Server-Sent Events）流式输出技术能够实现"边生成边展示"，显著提升用户体验。

本文将以本项目的知识库问答和 RAG 聊天功能为例，详细讲解 SSE 流式输出的实现原理、与 WebSocket 的对比，以及在实际项目中的最佳实践。

SSE 简介

SSE（Server-Sent Events） 是一种基于 HTTP 的服务器推送技术，允许服务器向客户端单向发送事件流。它是 HTML5 标准的一部分，与传统轮询相比，SSE 具有实时性强、延迟低、资源消耗少等优势。

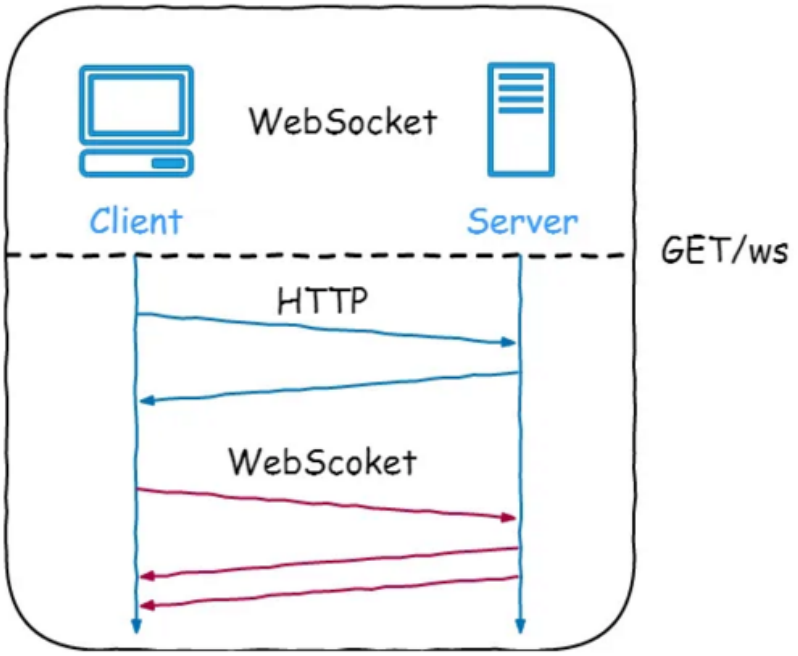
:authority:	chat.openai.com
:method:	POST
:path:	/backend-api/conversation
:scheme:	https
Accept:	text/event-stream
Accept-Encoding:	gzip, deflate, br
Accept-Language:	en-US
Authorization:	Bearer
	eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCIsImtpZCI6Ik1UaEVOVUpHTkVNMVFURTRNMEZ

SSE 的核心特点：

- **单向通信**：服务器向客户端推送数据
- **自动重连**：浏览器断开时会自动尝试重连
- **文本格式**：使用 UTF-8 编码的文本数据
- **标准协议**：基于 HTTP/HTTPS，无需额外协议升级

SSE 和 WebSocket 对比

WebSocket 是一种在 TCP 连接上进行全双工通信的协议，建立客户端和服务器之间的通信渠道。浏览器和服务器仅需一次握手，两者之间就直接可以创建持久性的连接，并进行双向数据传输。



SSE 与 WebSocket 作用相似，都可以建立服务端与浏览器之间的通信，实现服务端向客户端推送消息，但还是有些许不同：

- SSE 是基于 HTTP 协议的，它们不需要特殊的协议或服务器实现即可工作；WebSocket 需单独服务器来处理协议。
- SSE 单向通信，只能由服务端向客户端单向通信；WebSocket 全双工通信，即通信的双方可以同时发送和接受信息。
- SSE 实现简单开发成本低，无需引入其他组件；WebSocket 传输数据需做二次解析，开发门槛高一些。
- SSE 默认支持断线重连；WebSocket 则需要自己实现。
- SSE 只能传送文本消息，二进制数据需要经过编码后传送；WebSocket 默认支持传送二进制数据。

SSE 与 WebSocket 该如何选择？

- **选择 SSE**：服务器单向推送数据，如实时通知、AI 流式生成。
- **选择 WebSocket**：需要双向通信，如聊天室、协作编辑。

SSE 协议格式

SSE 是一组基于 **UTF-8** 编码的纯文本流。每一个事件（Message）由一行或多行设置组成，最后必须以一个**空行（两个换行符 `\n\n`）**作为该事件的结束标志。

在同一个事件块内，各个字段（data, event, id, retry）的顺序没有严格要求，但它们必须紧挨着，中间不能有空行：

```
data: 第一行内容
data: 第二行内容
data: 第三行内容

id: 事件ID
event: 自定义事件类型
retry: 3000

data: 下一个事件
```

字段说明：

字段	说明
`data`	具体的业务数据。若有多行，自动拼接。
`event`	自定义事件类型（默认为 `message`）。
`id`	事件 ID，用于断线重连后恢复。
`retry`	告诉客户端如果连接掉线，等待多少毫秒后再尝试重连。
`\n\n`	事件分隔符（两个换行符分隔不同事件）。

Spring Boot SSE 实现

Spring Boot 支持通过 `MediaType.TEXT_EVENT_STREAM_VALUE` 和 `Flux<ServerSentEvent<T>>` 实现流式输出。本项目使用的是 **WebMVC + Reactor 混合模式**：依赖 `spring-boot-starter-webmvc`，但通过 Spring AI 传递依赖的 `reactor-core` 来使用 `Flux` 进行 SSE 输出。

> **WebMVC vs WebFlux**：WebFlux 是完全响应式的非阻塞架构，而 WebMVC + Flux 只是在 Controller 层使用响应式类型做 SSE 输出，底层仍是阻塞式 Servlet 模型。对于 AI 流式输出场景，这种混合模式已足够使用。

基础用法

```java
@GetMapping(value = "/stream", produces = MediaType.TEXT_EVENT_STREAM_VALUE)
public Flux<String> streamData() {
    return Flux.interval(Duration.ofMillis(500))
        .map(seq -> "data: 消息 " + seq + "\n\n");
}
```

代码说明：

- `MediaType.TEXT_EVENT_STREAM_VALUE`：声明响应类型为 SSE 事件流，值为 `text/event-stream`。
- `Flux<String>`：响应式流类型，返回多个字符串事件。
- `Flux.interval()`：每 500 毫秒发射一个递增序列。
- 手动拼接 SSE 格式：`data:` 前缀 + `\n\n` 分隔符。

使用 ServerSentEvent 包装

```
@GetMapping(value = "/stream", produces = MediaType.TEXT_EVENT_STREAM_VALUE)
public Flux<ServerSentEvent<String>> streamData() {
    return Flux.interval(Duration.ofMillis(500))
        .map(seq -> ServerSentEvent.<String>builder()
            .id(String.valueOf(seq))           // 事件 ID
            .event("custom-event")              // 自定义事件类型
            .data("消息 " + seq)                // 事件数据
            .retry(3000L)                       // 重连间隔
            .build());
}
```

代码说明：

- `ServerSentEvent<T>`：Spring 提供的 SSE 事件包装类，自动处理 SSE 格式。
- 字段说明：
  - `id()`：设置事件唯一标识，客户端断线重传时可作为断点恢复的依据。
  - `event()`：自定义事件类型，客户端可通过 `addEventListener('custom-event')` 监听。
  - `retry()`：告诉客户端连接断开后等待多久重连（毫秒）。

Spring AI 流式调用

Spring AI 的 `ChatClient` 提供了流式调用接口：

```
Flux<String> responseFlux = chatClient.prompt()
    .system(systemPrompt)
    .user(userPrompt)
    .stream()  // 关键：使用 stream() 方法启用流式输出
    .content();
```

流式调用特点：

- **响应式编程**：返回 `Flux<String>`，支持背压控制。
- **增量返回**：每个 token 立即返回，不等待完整生成。
- **自动重试**：可配置重试策略。
- **超时控制**：可设置超时时间。

SSE 格式转义处理

SSE 协议使用 `\n\n` 作为事件分隔符，如果数据中包含换行符，会破坏协议格式：

```
# 错误示例：数据中的换行符破坏了 SSE 格式
data: 第一行内容
仍然在第一行              ← 被误认为是新的事件
data: 第二行内容

# 正确示例：转义换行符
data: 第一行内容\n仍然在第一行
data: 第二行内容\n
```

后端转义实现：

```
.map(chunk -> ServerSentEvent.<String>builder()
    .data(chunk.replace("\n", "\\n").replace("\r", "\\r"))
    .build())
```

前端反向转义：

```
const text = chunk.replace(/\\n/g, '\n').replace(/\\r/g, '\r');
```

错误处理与性能优化

超时控制

```
return responseFlux
    .timeout(Duration.ofSeconds(30))
    .onErrorResume(TimeoutException.class, e -> {
        log.error("流式输出超时", e);
        return Flux.just("【错误】回答生成超时，请缩短问题或稍后重试。");
    });
```

背压控制

在 WebMVC + Reactor 混合模式下，`Flux` 的背压机制仍然有效，可以控制数据流速度：

```
Flux<String> responseFlux = chatClient.prompt()
    .stream()
    .content()
    .onBackpressureBuffer();  // 缓冲背压
```

> **注意**：虽然 Reactor 的背压机制可用，但 WebMVC 底层仍是阻塞式 Servlet 线程模型，连接数受限于 Servlet 容器的线程池配置（如 Tomcat 的 `server.tomcat.threads.max`）。

Servlet 容器配置

本项目使用 **虚拟线程**（Java 21+），无需担心 SSE 长连接占用线程池：

```
spring:
  threads:
    virtual:
      enabled: true  # 启用虚拟线程
```

虚拟线程 vs 平台线程：

场景	平台线程	虚拟线程
200 并发 SSE	线程池满，排队	轻松处理
AI 调用等待 3 秒	线程阻塞，占用资源	自动挂起，让出资源
10000 并发请求	拒绝服务	正常处理

> **为什么不用 WebFlux?** 虚拟线程只需一行配置，无需重写代码；而 WebFlux 需要将 JPA 换成 R2DBC，改造成本极高。对于 I/O 密集型的 AI 应用，虚拟线程是更务实的选择。

关于虚拟线程的详细介绍可以看这篇文章：虚拟线程常见问题总结。

超时设置

```yaml
spring:
  ai:
    openai:
      chat:
        options:
          duration: 30s  # AI 调用超时时间
```

最佳实践

SSE 格式规范

SSE 对格式极其敏感，任何微小的格式偏差都可能导致客户端解析器挂起。

规范项	说明	项目实践
数据转义	`data` 字段内的原始数据若包含 `\n`，必须转义为 `\\n`	RAG 聊天使用 `chunk.replace("\\n", "\\\\n")` 转义
事件定界	严格使用 `\n\n`（双换行）作为每个事件块的结束标志	使用 `ServerSentEvent` 自动处理
字符编码	强制使用 UTF-8	Spring Boot 默认使用 UTF-8
心跳机制	每隔 15-30 秒发送冒号开头的注释行（如 `: heartbeat\n\n`）	本项目暂未实现，生产环境建议添加

稳定性与异常处理

SSE 是长连接，资源释放和状态一致性是重中之重。

策略维度	核心操作	项目实践
消息占位	流式开始前先创建消息占位，完成后更新内容	`prepareStreamMessage()` + `completeStreamMessage()`
状态管理	使用 `completed` 字段区分"生成中"和"已完成"	前端根据此字段显示 loading 状态
异常保存	无论成功或失败，都保存已接收的内容	`doOnError` 中保存 `fullContent`
超时降级	设置合理的超时时间，超时后返回友好提示	`.timeout(Duration.ofSeconds(30))` + `onErrorResume`

RAG 聊天流式处理示例：

```
return sessionService.getStreamAnswer(sessionId, request.question())
    .doOnNext(fullContent::append)  // 累积完整内容
    .map(chunk -> ServerSentEvent.<String>builder()
        .data(chunk.replace("\n", "\\n").replace("\r", "\\r"))  // 转义换行符
        .build())
    .doOnComplete(() -> {
        // 成功：更新消息内容并标记为已完成
        sessionService.completeStreamMessage(messageId, fullContent.toString());
    })
    .doOnError(e -> {
        // 失败：保存已接收的部分内容
        String content = !fullContent.isEmpty()
            ? fullContent.toString()
            : "【错误】回答生成失败：" + e.getMessage();
        sessionService.completeStreamMessage(messageId, content);
    });
```

前端性能优化

流式数据高频更新会导致页面卡顿，需要优化渲染策略。

```
// 使用 requestAnimationFrame + React Transition 优化渲染
let fullContent = '';
const rafRef = useRef<number>();

onMessage((chunk: string) => {
  fullContent += chunk;
  if (rafRef.current) {
    cancelAnimationFrame(rafRef.current);  // 取消上一次未执行的渲染
  }
  rafRef.current = requestAnimationFrame(() => {
    startTransition(() => {
      updateAssistantMessage(fullContent);
    });
  });
});
```

优化要点：

- **requestAnimationFrame**：将渲染推迟到下一帧，避免高频更新
- **cancelAnimationFrame**：取消上一次未执行的渲染，只保留最新一次
- **startTransition**：React 18 并发特性，将渲染标记为低优先级

Nginx 代理配置

如果使用 Nginx 反向代理，必须关闭缓冲配置：

```
location /api/ {
    proxy_pass http://backend;
    proxy_buffering off;      # 关闭代理缓冲
    proxy_cache off;          # 关闭代理缓存
    proxy_read_timeout 300s;  # 读取超时时间
    proxy_set_header Connection '';     # 清空 Connection 头
    proxy_set_header Cache-Control 'no-cache';  # 禁止缓存
}
```

配置说明：

- `proxy_buffering off`：禁用缓冲，确保数据实时转发
- `proxy_cache off`：禁用缓存，避免旧数据被缓存
- `proxy_read_timeout`：设置合理的读取超时时间
- `Connection ''`：清空 Connection 头，保持长连接

总结

SSE 流式输出是提升 AI 应用用户体验的关键技术。核心要点：

要点	说明
协议选择	单向推送选 SSE，双向通信选 WebSocket
协议格式	`data:` 前缀 + `\n\n` 分隔符
格式转义	换行符 `\n` → `\\n`，避免破坏事件分隔符
消息持久化	先创建占位（`completed=false`），完成后更新内容
限流保护	使用 `@RateLimit` 注解防止 AI 服务被刷爆
异常处理	无论成功失败都保存已接收内容，避免数据丢失
前端优化	使用 RAF + React Transition 优化高频渲染

项目核心文件：

- `modules/knowledgebase/KnowledgeBaseController.java` - 知识库控制器
- `modules/knowledgebase/RagChatController.java` - RAG 聊天控制器
- `modules/knowledgebase/service/KnowledgeBaseQueryService.java` - 查询服务
- `modules/knowledgebase/service/RagChatSessionService.java` - 会话管理服务
- `frontend/src/api/knowledgebase.ts` - 知识库前端 API
- `frontend/src/api/ragChat.ts` - RAG 聊天前端 API